Dodona 01 — cluster sanity

go / no-go, no load 2026-10-10 18:28 UTC

Objective

Does the cluster serve a real completion, and does it go THROUGH the control plane rather than past it?

Falsified by: orch_requests_total not moving, or a pool not answering

The promises everything later is scored against

SLO	target	direction
TTFT p95	1.5s	max
turn p95	30s	max
preemptions	0	max
shed rate at target load	0.05	max
concurrent sessions	derived in 02	min

From benchmarking/slos.py. The session target is derived in 02.

Concurrency target not measured yet — run 02-capacity.

1. Is anything listening?

PASS gateway /metrics

PASS gateway /v1/models

PASS prefill /metrics

PASS decode /metrics

>> All four endpoints answer, so the readings below are real.

2. Which pool does which job?

PASS gateway advertises: Qwen/Qwen2.5-7B-Instruct-AWQ

pool	running	waiting	prompt tok	gen tok
prefill	0	0	353,311	129
decode	0	0	353,311	258

prompt tok of 0 on a pool means nothing was ever routed to it.

PASS prefill emits 1.00 tokens per request (max_tokens=1 means ~1.0)

>> The split is real: prefill emitted 129 tokens over 129 requests — 1.00 each, which is the max_tokens=1 leg doing prompt work only, against decode's 258. But both pools show the same prompt-token count, so decode is still paying to process the prompt — section 5 measures why.

3. How much room does one pod have?

quantity	value	where it comes from
slice x util	10.80 GiB	HAMi gpumem x --gpu-memory-utilization
weights	5.20 GiB	Qwen2.5-7B AWQ int4, embeddings in fp16
activations + CUDA graphs	1.50 GiB	budget minus weights minus KV
KV cache	4.10 GiB	pod log: Available KV cache memory

```
PASS  a pool of 4.7 is small enough for KV to bind
```

>> KV gets 4.10 of 10.80 usable GiB, which buys 4.7 full-length sequences per pod and a pool of 4.7. That is the scarcity the admission experiments need. The whole card would suggest 73.5 — do not quote that for a slice.

4. Does a request go through the gateway?

```
PASS  orch_requests_total 129 -> 130
PASS  completion in 0.331s
```

field	value
content	'ok'
via	local
latency	0.331s (tunnel included)

>> The counter moved and the model answered 'ok'. Guardrails, admission, placement and the hop all ran. via=local means our cluster served it rather than bursting to a provider.

5. Did any KV actually move?

```
PASS  the router RECORDED a hop
```

field	value
src	prefill-0
dst	decode-0
req_id	http
tokens	16
backend	mooncake
prefix	802848469d1623df

a record, not evidence of bytes

where decode's prompt KV came from	tokens
computed from scratch	0
decode's own prefix cache	310,864
the connector (a real hop)	42,483

decode is SENT the whole prompt either way, so prompt_tokens_total never moves — only this split does.

connector counter	value
times asked	42,483
times it returned blocks	42,483

```
PASS  the connector has delivered at least one block
```

>> The connector delivered 42,483 tokens of KV. The hop is real and section 8 should show a flat latency curve.

6. Is anything scarce right now?

pool	kv_cache_usage_perc	vs 0.90
prefill	0.0000	below
decode	0.0000	below

>> Peak KV usage is 0.0% on an idle cluster, so nothing is scarce and the policy has nothing to decide. That is expected here. 02-capacity finds the load at which this crosses 90%, and until it does every admission arm would measure an inert component.

7. Latency on a small request

metric	seconds
n	10
min	0.319
p50	0.340
p95	0.344
max	0.344

>> p50 0.340s, p95 0.344s, spread 0.025s across ten requests, drifting +2% end to end. Stable. The tunnel is inside these numbers, so use make load to quote any.

8. Does decode pay for the prompt?

prompt tokens	p50 seconds
256	0.349
1,024	0.395
4,096	0.396
8,192	0.466
12,288	0.524

PASS all 5 prompt lengths completed

PASS latency grew 1.5x for 48x more prompt (sub-linear means the prompt is not being recomputed in fu

>> From 256 to 12,288 prompt tokens (48x), p50 went 0.349s to 0.524s (1.5x). Section 5 says the connector delivered 42,483 tokens, so this slope is the cost of MOVING the KV, not of recomputing it. A transfer is not free.

9. Grafana

vLLM — KV, running/waiting, TTFT, prefix cache

http://127.0.0.1:31495/d/dodona-vllm?from=now-30m&to=now

Gateway — admit, shed, place, latency by stage

```
http://127.0.0.1:31495/d/dodona-gateway?from=now-30m&to=now
```

Router — pick, sticky, KV hop

```
http://127.0.0.1:31495/d/dodona-router?from=now-30m&to=now
```

KEDA — scaler value against replicas

```
http://127.0.0.1:31495/d/dodona-keda?from=now-30m&to=now
```

HAMi — device memory against the slice cap

```
http://127.0.0.1:31495/d/dodona-hami?from=now-30m&to=now
```

explore: KV usage per pod

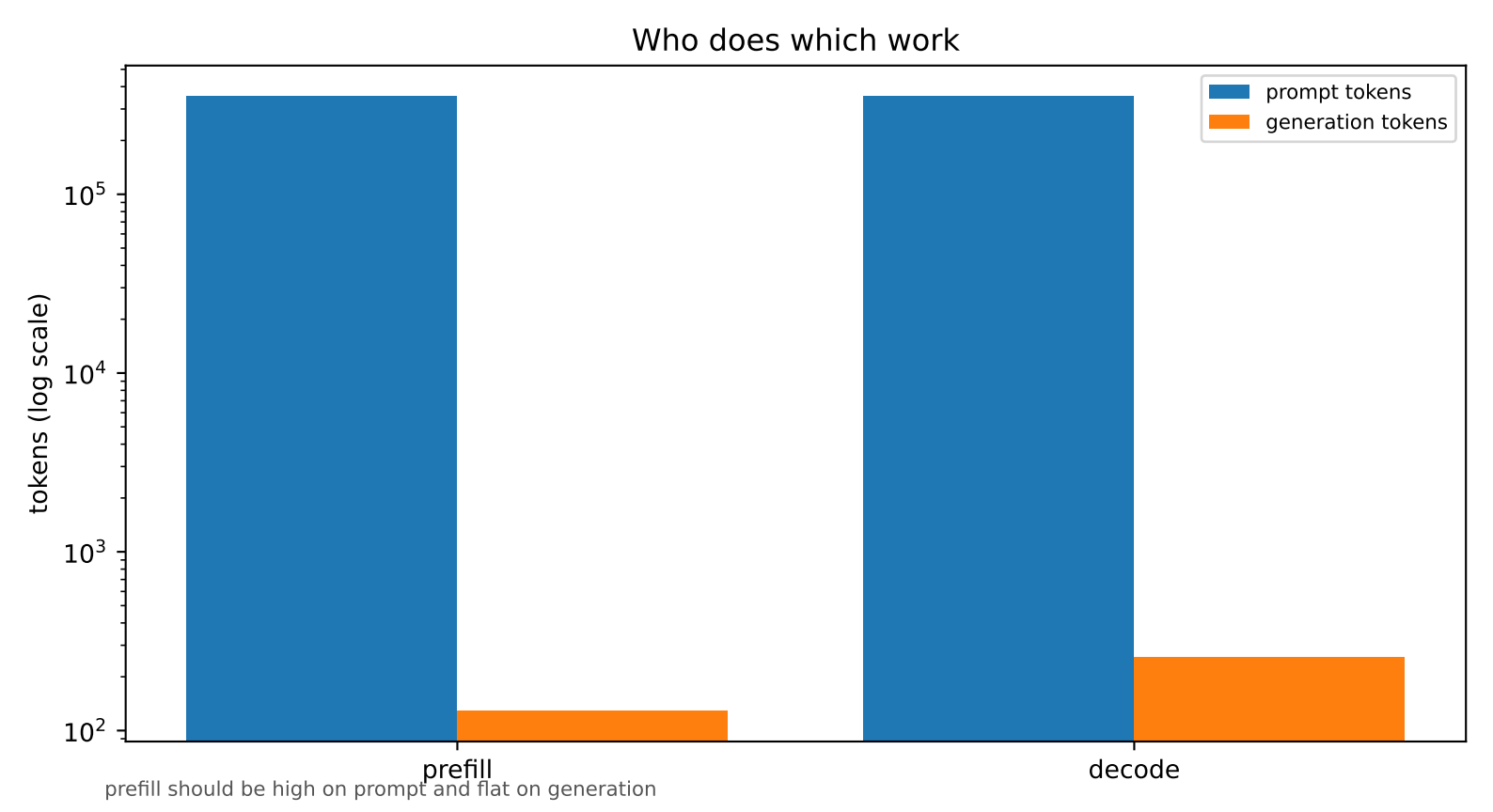
```
http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22
```

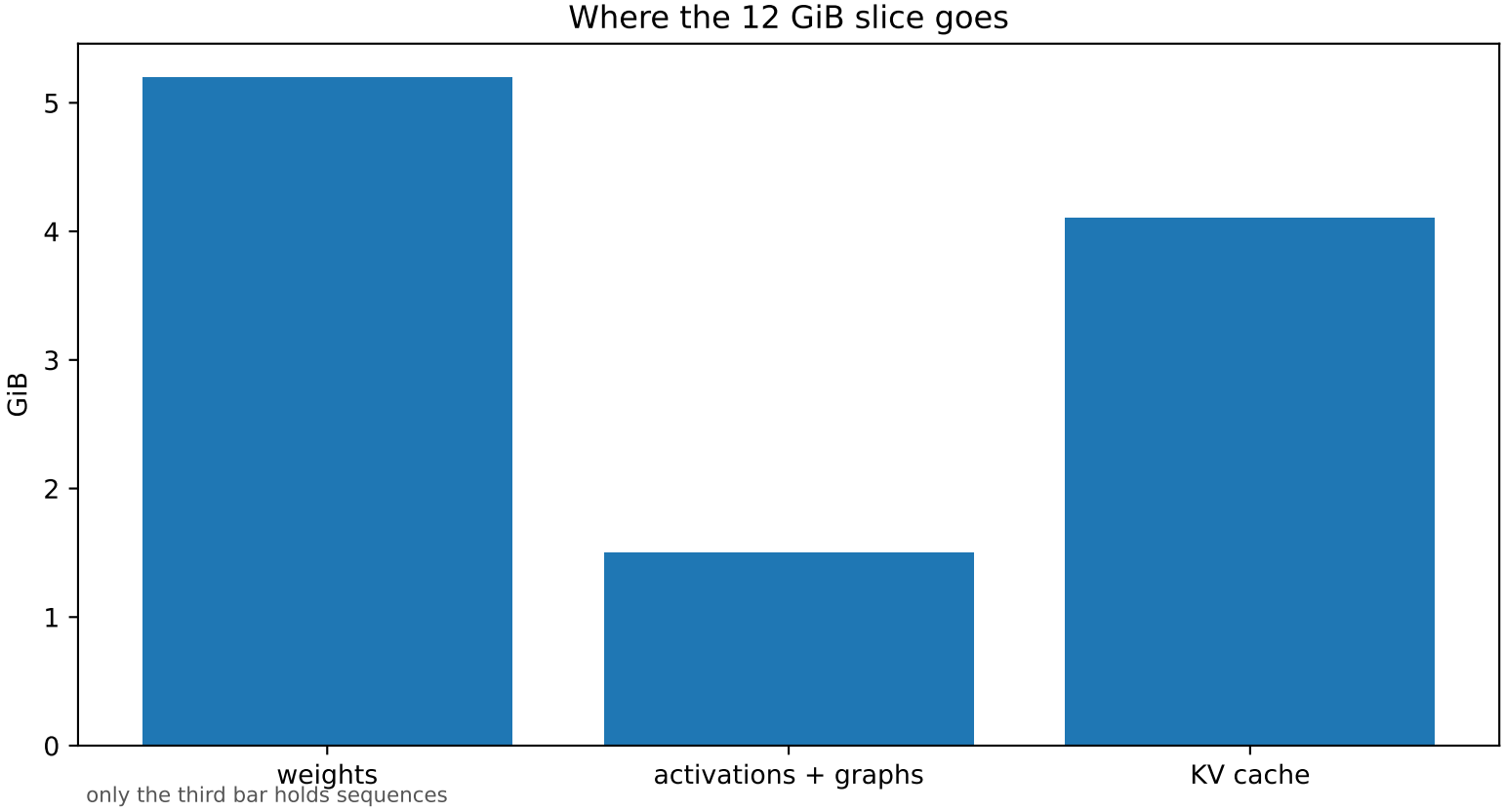
explore: queue depth per pod

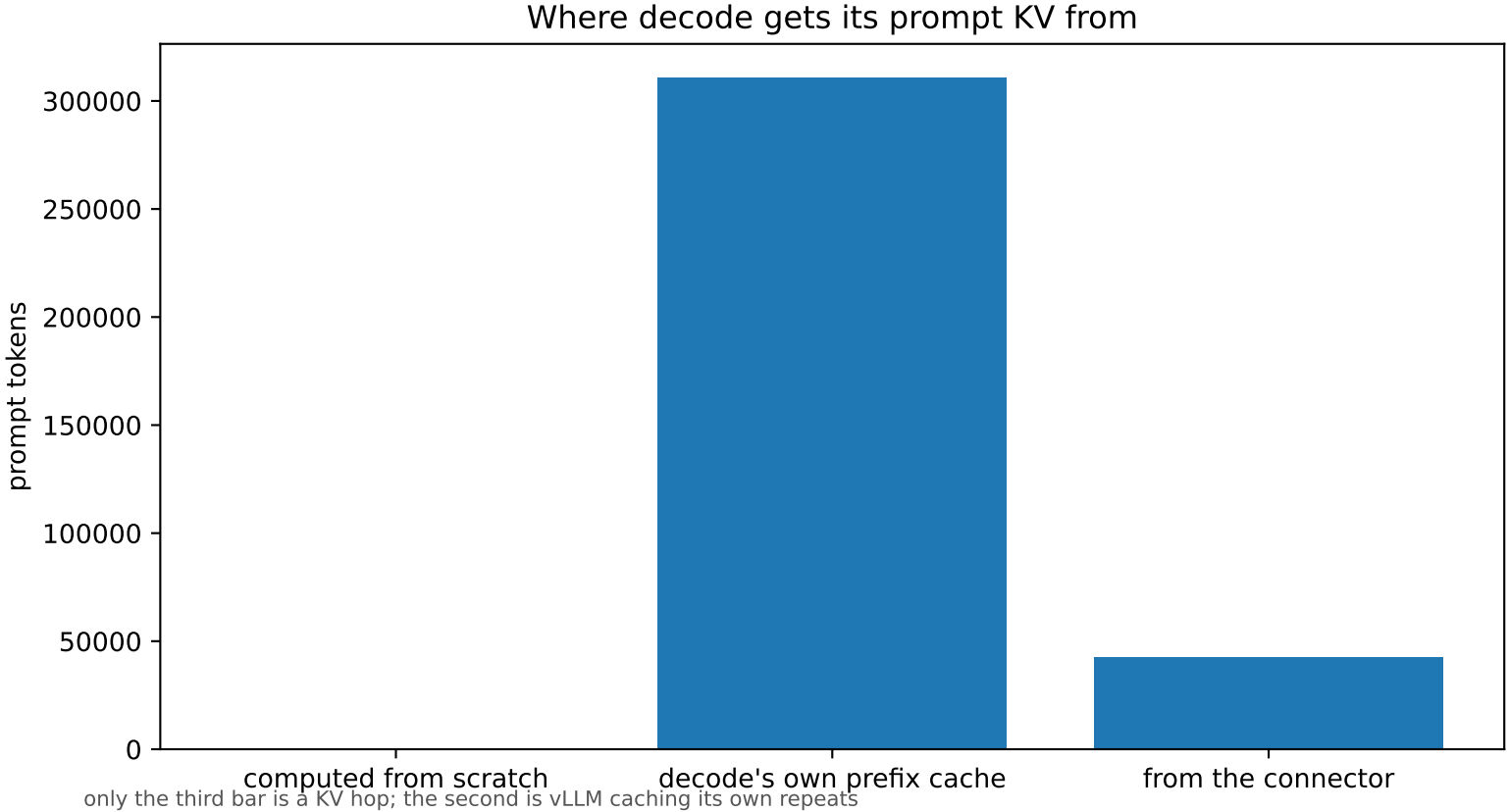
```
http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22
```

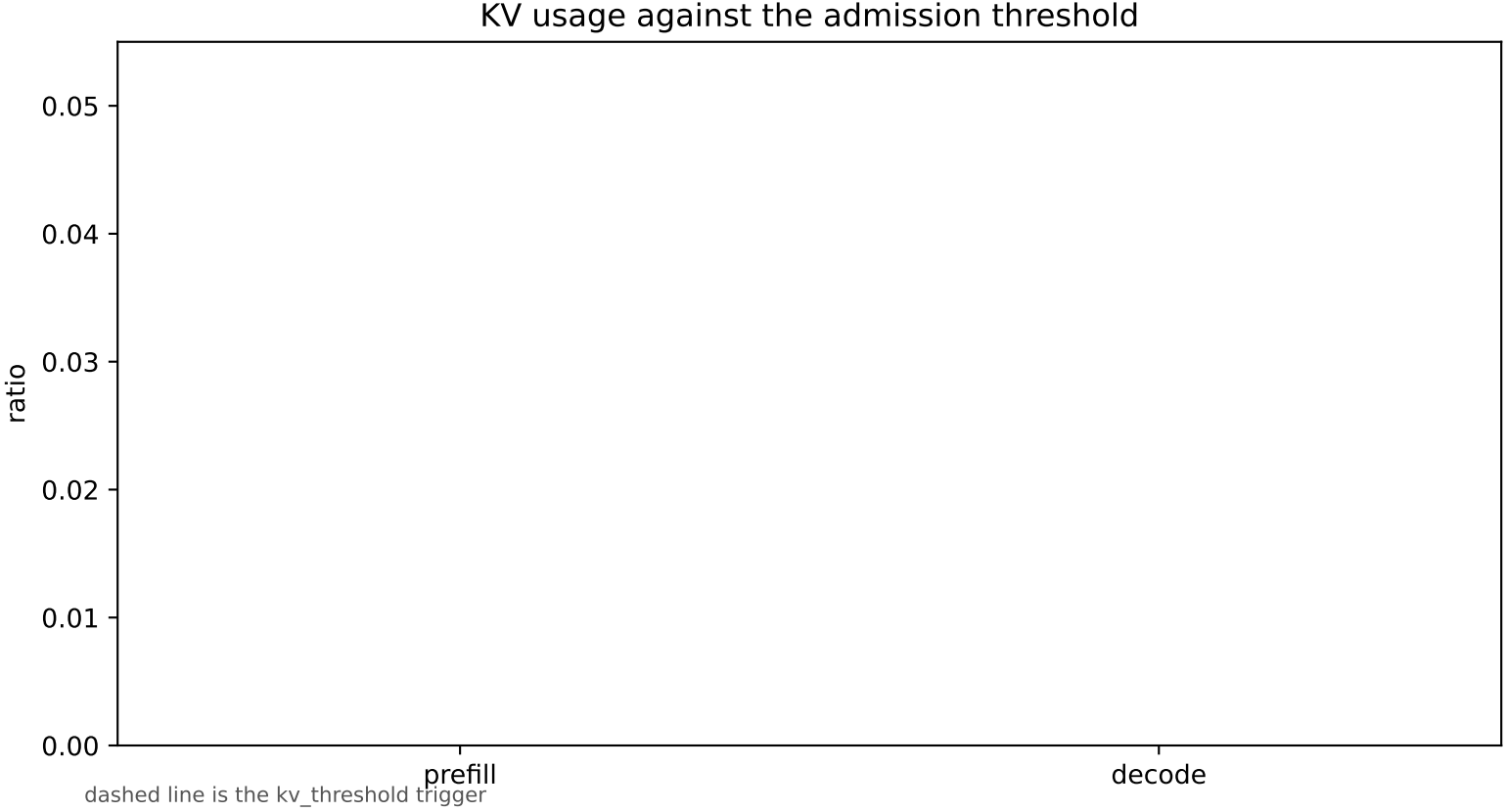
explore: KV actually transferred

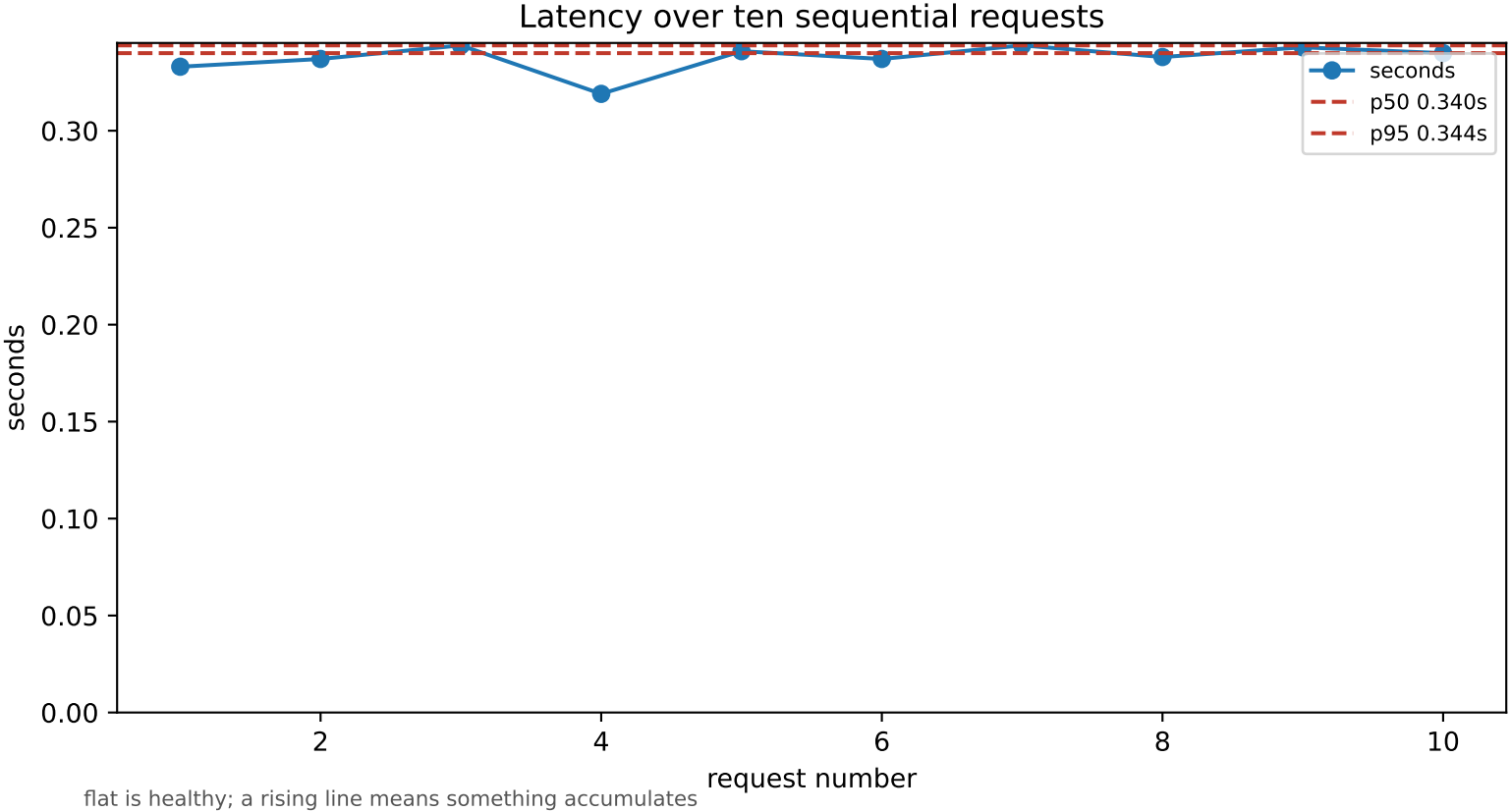
```
http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22
```


Who does which work
prompt tokens
generation tokens
tokens (log scale)
10^5
10^4
10^3
10^2
prefill
decode
prefill should be high on prompt and flat on generation

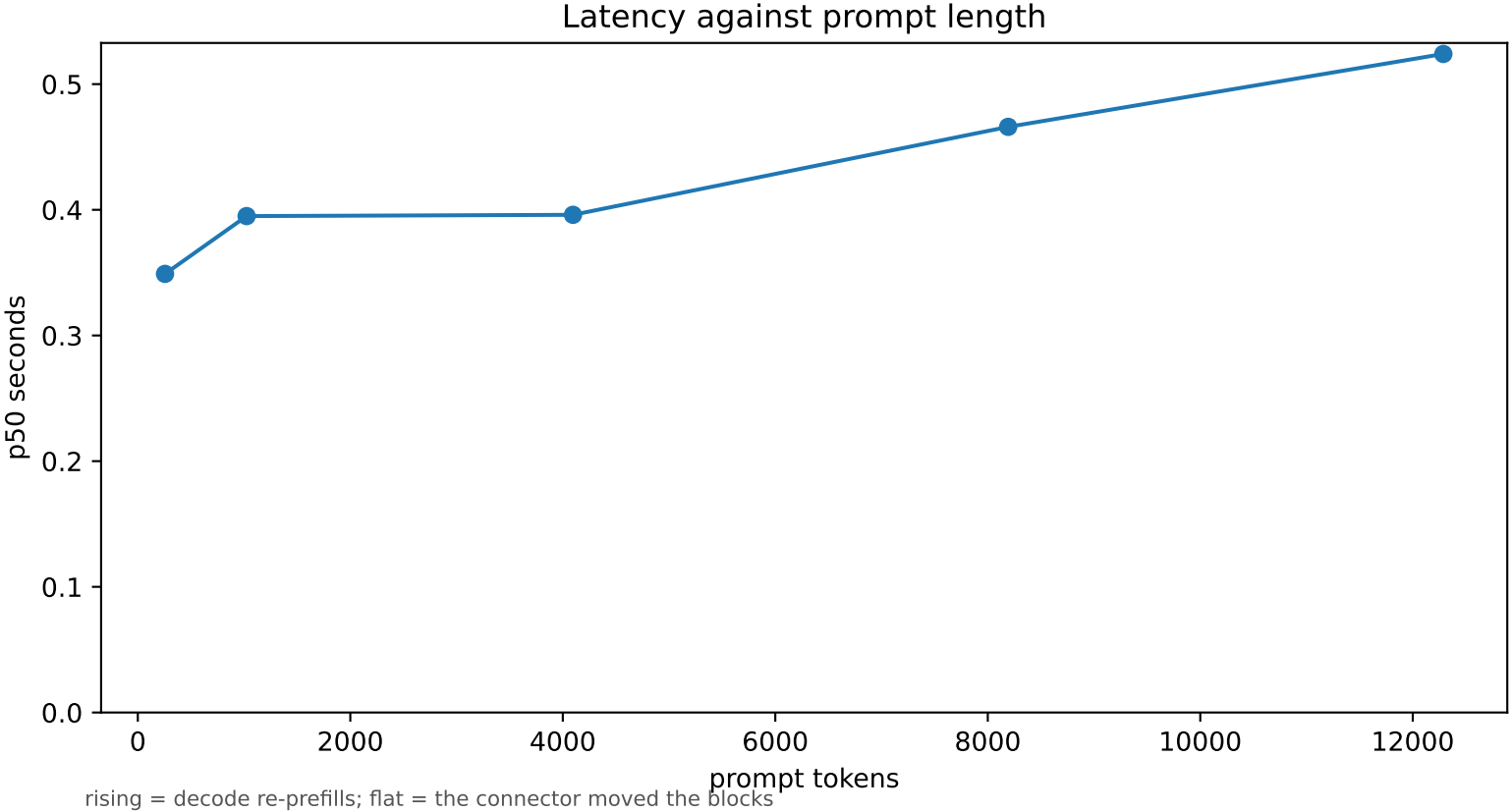

Where the 12 GiB slice goes
GiB
0
1
2
3
4
5
weights
activations + graphs
KV cache
only the third bar holds sequences


Where decode gets its prompt KV from
prompt tokens
300000
250000
200000
150000
100000
50000
0
computed from scratch
decode's own prefix cache
from the connector
only the third bar is a KV hop; the second is vLLM caching its own repeats


KV usage against the admission threshold
ratio
0.05
0.04
0.03
0.02
0.01
0.00
prefill
decode
dashed line is the kv_threshold trigger



flat is healthy; a rising line means something accumulates


Latency against prompt length
p50 seconds
0.0
0.1
0.2
0.3
0.4
0.5
0
2000
4000
6000
8000
10000
12000
prompt tokens
rising = decode re-prefills; flat = the connector moved the blocks